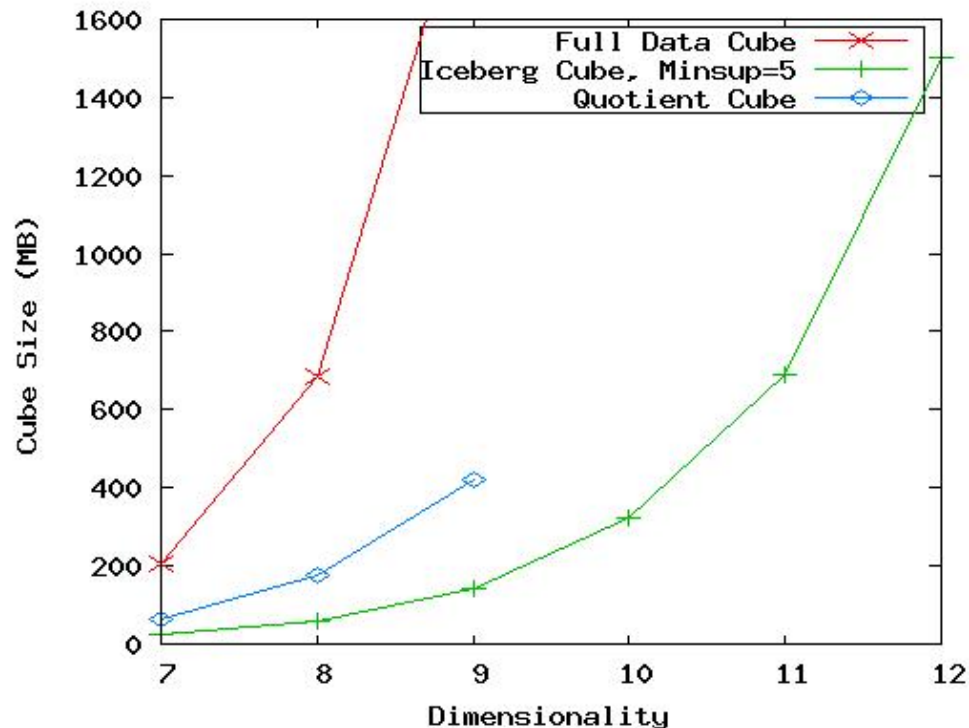


The Curse of Dimensionality

- None of the previous cubing method can handle high dimensionality!
- A database of 600k tuples. Each dimension has cardinality of 100 and *zipf* of 2.



Motivation of High-D OLAP

- Challenge to current cubing methods:
 - The “curse of dimensionality” problem
 - Iceberg cube and compressed cubes: only delay the inevitable explosion
 - Full materialization: still significant overhead in accessing results on disk
- High-D OLAP is needed in applications
 - Science and engineering analysis
 - Bio-data analysis: thousands of genes
 - Statistical surveys: hundreds of variables

Fast High-D OLAP with Minimal Cubing

- Observation: OLAP occurs only on a small subset of dimensions at a time
- Semi-Online Computational Model
 1. Partition the set of dimensions into **shell fragments**
 2. Compute data cubes for each shell fragment while retaining **inverted indices** or **value-list indices**
 3. Given the pre-computed **fragment cubes**, dynamically compute cube cells of the high-dimensional data cube *online*

Properties of Proposed Method

- Partitions the data vertically
- Reduces high-dimensional cube into a set of lower dimensional cubes
- Online re-construction of original high-dimensional space
- Lossless reduction
- Offers tradeoffs between the amount of pre-processing and the speed of online computation

Example Computation

- Let the cube aggregation function be `count`

<i>tid</i>	A	B	C	D	E
1	a1	b1	c1	d1	e1
2	a1	b2	c1	d2	e1
3	a1	b2	c1	d1	e2
4	a2	b1	c1	d1	e2
5	a2	b1	c1	d1	e3

- Divide the 5 dimensions into 2 shell fragments:
 - (A, B, C) and (D, E)

1-D Inverted Indices

- Build traditional invert index or RID list

Attribute Value	TID List	List Size
a1	1 2 3	3
a2	4 5	2
b1	1 4 5	3
b2	2 3	2
c1	1 2 3 4 5	5
d1	1 3 4 5	4
d2	2	1
e1	1 2	2
e2	3 4	2
e3	5	1

Shell Fragment Cubes

- Generalize the 1-D inverted indices to multi-dimensional ones in the data cube sense

Cell	Intersection	TID List	List Size
a1 b1	1 2 3 $\cap$ 1 4 5	1	1
a1 b2	1 2 3 $\cap$ 2 3	2 3	2
a2 b1	4 5 $\cap$ 1 4 5	4 5	2
a2 b2	4 5 $\cap$ 2 3	$\otimes$	0

Shell Fragment Cubes (2)

- Compute all cuboids for data cubes ABC and DE while retaining the inverted indices
- For example, shell fragment cube ABC contains 7 cuboids:
 - A, B, C
 - AB, AC, BC
 - ABC
- This completes the offline computation stage

Shell Fragment Cubes (4)

- Shell fragments do not have to be disjoint
- Fragment groupings can be arbitrary to allow for maximum online performance
 - Known common combinations (e.g., <city, state>) should be grouped together.
- Shell fragment sizes can be adjusted for optimal balance between offline and online computation

ID_Measure Table

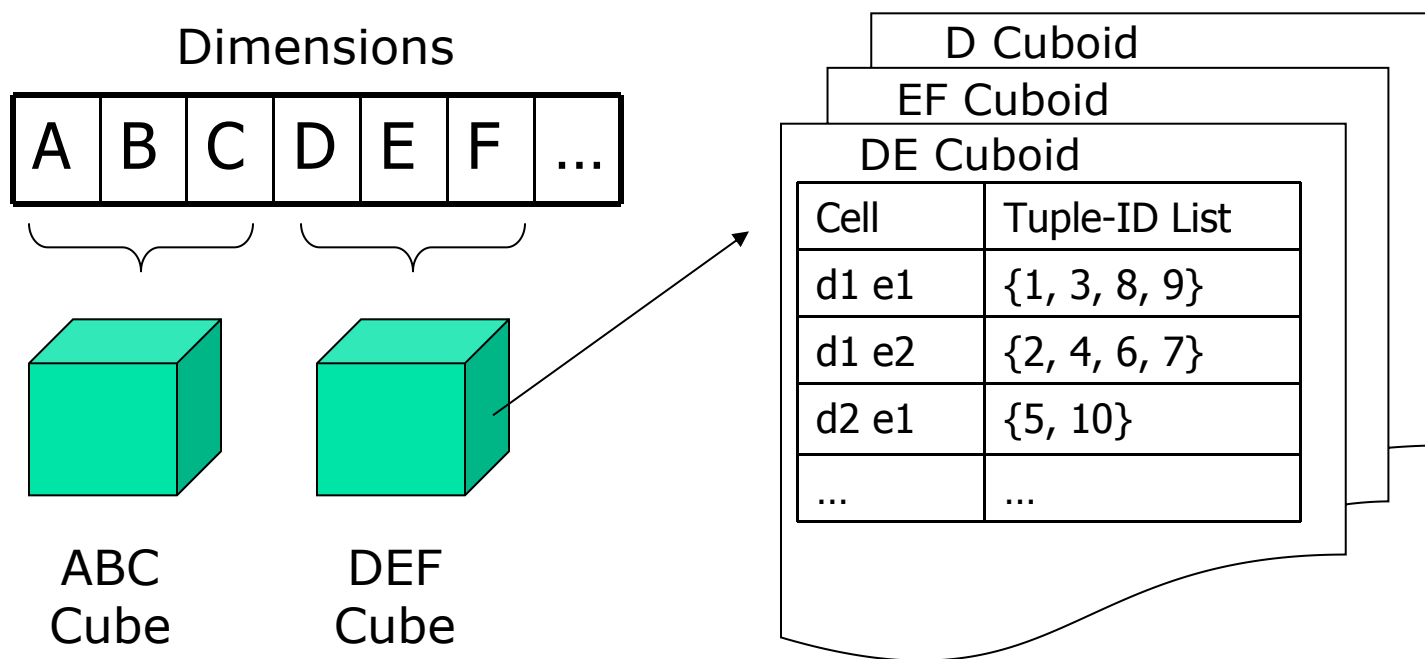
- If measures other than `count` are present, store in *ID_measure* table separate from the shell fragments

tid	count	sum
1	5	70
2	3	10
3	8	20
4	5	40
5	2	30

The Frag-Shells Algorithm

1. Partition set of dimension $(A_1, \dots, A_n)$ into a set of k fragments $(P_1, \dots, P_k)$.
2. Scan base table once and do the following
 3. insert $\langle \text{tid}, \text{measure} \rangle$ into ID_measure table.
 4. for each attribute value a_i of each dimension A_i
 5. build inverted index entry $\langle a_i, \text{tidlist} \rangle$
6. For each fragment partition P_i
 7. build local fragment cube S_i by intersecting tid-lists in bottom-up fashion.

Frag-Shells (2)



Online Query Computation

- For example, returns a 2-D data cube.

$$(a_1, b_1, c_1, *, e_1) \quad \langle a_1, a_2, K, a_n : M \rangle$$

Online Query Computation (2)

- Given the fragment cubes, process a query as follows
 1. Divide the query into fragment, same as the shell
 2. Fetch the corresponding TID list for each fragment from the fragment cube
 3. Intersect the TID lists from each fragment to construct **instantiated base table**
 4. Compute the data cube using the base table with any cubing algorithm

Online Query Computation (3)

